University of California

Santa Barbara

Leveraging WebAssembly as a Universal Distributable for Embedded Audio Applications

A project submitted in partial satisfaction

of the requirements for the degree

Master of Science

in

Media Arts and Technology

by

Joel A. Jaffe

Committee in charge:

Professor Jennifer Jacobs, Chair

Professor Curtis Roads

Dr. Karl Yerkes

June 2026

The project of Joel A. Jaffe is approved.

Professor Curtis Roads

Dr. Karl Yerkes

Professor Jennifer Jacobs, Chair

May 2026

Leveraging WebAssembly as a Universal Distributable for Embedded Audio Applications

Copyright © 2026

by

Joel A. Jaffe

Acknowledgements

The author thanks their committee and the UCSB community at large for supporting and guiding them throughout their studies.

Abstract

Leveraging WebAssembly as a Universal Distributable for Embedded Audio Applications

by

Joel A. Jaffe

Third-party plugins are ubiquitous in desktop audio software, but remain rare in embedded systems. Driven by the vast heterogeneity of embedded devices, siloed platforms have become the norm, with each hardware manufacturer maintaining proprietary digital signal processing (DSP) modules that are incompatible, even when offering many of the same algorithms to end users.

To enable third-party DSP in embedded systems, this work investigates the WebAssembly (WASM) binary instruction format as a hardware-agnostic intermediate representation (IR) for distributing third-party audio processing.

To assess feasibility under real-time constraints, representative audio DSP algorithms are compiled to WASM and compared across three execution strategies on an embedded platform (Daisy Seed). Performance measurements demonstrate that runtimes leveraging ahead-of-time (AOT) translation of WASM yield acceptable performance for a meaningful subset of audio processing algorithms, indicating the viability of WASM as a portable distributable for embedded DSP.

In addition to embedded deployment, WASM is supported by runtimes in desktop audio, plugin hosts, and web environments, enabling shared DSP implementations and presets across platforms. To support further work in unifying these environments, this dissertation also documents the creation of an online development tool for authoring and deploying WASM-based audio plugins.

Contents

1 Introduction **1**

1.1 Embedded Audio Systems 2

1.2 WebAssembly as an Intermediate Representation 3

2 Related Work **5**

2.1 Single-source, Multi-target Audio DSP 5

2.2 WebAssembly in IoT Development 7

2.3 Existing Approaches to Third-Party Plugins in Embedded Audio 8

3 Performance Benchmarking **11**

3.1 Algorithms 11

3.1.1 FAUST Reverb 11

3.1.2 Gen~Phaser 12

3.1.3 Klon Centaur 12

3.1.4 Neural Amp Modeler 12

3.2 Runtimes 13

3.2.1 Wasm Micro Runtime (WAMR) 13

3.2.2 wasm2c 13

3.2.3 wasmi 13

3.3 Embedded Target 14

3.4 Benchmarking Code 14

4 Developer Tooling **16**

5 Discussion **19**

6 Conclusion **21**

List of Figures

2.1 *Schematic representation of an audio DSL or plugin framework targeting multiple deployment environments.* 6

2.2 *Schematic representation of per-language, per-target toolchains.* 6

2.3 *Schematic representation of multiple languages targeting an intermediate representation that supports multiple deployments.* 7

2.4 *The Darkglass Anagram, a commercial product that supports LV2 plugins* 9

3.1 *Performance comparison relative to native (1.0x).* 15

4.1 *DaisyOnline, an online IDE for developing and deploying WebAssembly on the Daisy Seed* 18

List of Tables

3.1 *Raw sample throughput (samples/sec) across runtimes. Cells shaded green represent algorithm/runtime combinations with throughput greater than 48kHz, indicating real-time performance* 14

3.2 *Relative performance comparison of different algorithms across runtimes.* 15

Chapter 1

Introduction

The landscape of audio software in 2026 is diverse. Deployments include standalone applications spanning various operating systems and computer architectures, myriad plugin formats for digital audio workstations (DAWs), game engine plugins, web audio, embedded systems, and mobile apps, to name a few. In many cases, interoperability between these environments is desirable from both end user and developer perspectives.

For users, interoperability promotes sonic consistency. For example, an electric instrumentalist may want the same effect processing they use in a DAW to also be available in a live performance context, ideally on hardware that is tailored to that environment. For developers, deployment to multiple targets from a single codebase and build system endows greater ability to reach more end users with their software.

The goal of this project is to investigate means of targeting myriad deployments from one (non-source) distributable provided by audio developers, a *universal distributable*. Such a deliverable can confer many advantages, but also carries with it some inherent downsides. Advantages include simplified build systems, easier porting of algorithms to new platforms, and greater interoperability between heterogeneous deployments. Downsides include performance degradation from the abstraction layer, increased complexity of debugging, and easier de-compilation. In this project I have focused on deployment of

such a universal distributable in embedded systems, as these resource-contained environments provide the greatest challenge in portability. If deployment to embedded systems proves feasible, then all other targets may follow suit with relative ease.

1.1 Embedded Audio Systems

Embedded systems are computer systems specialized for a particular task[6]. Examples that perform audio processing surround us: smart home devices, digital video cameras, and wireless earphones. Common form factors for musical contexts include sampling synthesizers, stompbox effect processors, and digital mixers.

Traditionally, these systems are developed with tightly controlled, manufacturer-specific software ecosystems[33]. Each hardware platform provides its own developer tooling, firmware architecture, and collection of DSP modules. While this tight coupling between software and hardware provides advantages in performance, it also produces ecosystem siloing. Algorithms are not easily portable across devices, and innovation is constrained by the resources and priorities of individual manufacturers.

This ecosystem siloing introduces several practical limitations. Most notable among them:

- Individual algorithm quality may suffer because hardware vendors are responsible for implementing and maintaining the broad range of processing that users expect, often redundantly *re*-implementing the popular modules offered by similar products.
- Users cannot easily maintain sonic consistency between desktop environments and embedded devices, as the robust world of third-party plugin software for desktop audio is not available in embedded contexts.
- Shifts in popular audio processing techniques create ongoing maintenance burdens. When new paradigms emerge, manufacturers must issue firmware updates or even redesign hardware to remain competitive.

Given the resource-constrained nature of embedded systems, the goal of using a unified distributable to address the above listed issues is lofty- but not unprecedented.

1.2 WebAssembly as an Intermediate Representation

In computing generally, intermediate representation (IR) often takes one of two forms. Some forms, such as LLVM IR[30], are used internally by compilers as a translation stage in the process of transforming high-level source code into architecture-specific machine code prior to program execution. Another form of IR is *bytecode*, which is also translated from high-level source code but is intended to run inside a *virtual machine*[34], a software program that translates IR to native machine code at runtime.

Recent Internet of Things (IoT) research[42][32][17] has explored WebAssembly (WASM) as an IR of both forms described above, with the general goal of WASM as a "write once, run anywhere" substrate for self-contained applications. A major advantage of WASM over other IRs is that the "write once" step can occur in many different programming languages, as the standard was developed from the onset to be language-agnostic[16].

Prior research in digital audio has proposed WASM as a universal distributable for audio DSP with a framework entitled *Audio Anywhere*[15], although development of this project has ceased. In their work, the authors explored interpreter-based execution and ahead-of-time (AOT) translation strategies for their runtimes targeting embedded, but limited their scope to WASM modules compiled from a single input source language. Building on their research, I have investigated alternative AOT translation strategies for WASM modules, enabled additional high-level source code languages (C/C++) as input, benchmarked a wider range of audio processing algorithms, and developed web-based tooling for development and distribution.

Inline with performance measurements presented in [15], my benchmark results indi-

cate that runtime interpretation of WASM is not a viable strategy for real-time embedded audio workloads. However, runtimes leveraging AOT translation of WASM modules yield acceptable performance under certain conditions, notably when a program's memory footprint is known at compile time. Performance may be further improved by better leveraging WASM's interface for importing functions from its host, greater optimization of compiler toolchains, and increased runtime specialization.

My findings suggest that with further research, a carefully specified WASM ABI for audio processing and parameter management could enable language-agnostic development of encapsulated audio DSP coupled with deployment across embedded devices, desktop applications, and web platforms from a universal distributable. In that aim, the primary contributions of this work are the exploration of multiple execution strategies for WASM-based audio DSP modules, empirical benchmarking of their relative performance on representative embedded hardware, and creation of web-based developer tooling to facilitate their authoring and deployment.

Chapter 2

Related Work

2.1 Single-source, Multi-target Audio DSP

"Write once, run anywhere" development and deployment of audio DSP algorithms is not a fundamentally new concept. Many solutions have arisen within the audio developer community, although none boast the advantages of a universal distributable.

One approach is the use of *intermediary plugin frameworks*, which have become very popular among audio developers[29]. Plugin frameworks provide a uniform application programming interface (API) in a particular programming language (typically C/C++) for producing many platform-specific executables from the same source code. The most widely used of these frameworks is JUCE[41], which has become the industry standard. JUCE provides a C++ API by which the same audio processing and user interface code can be compiled for multiple targets, including cross-platform standalone applications, myriad plugin formats for digital audio workstations (DAWs), game engine plugins, and mobile apps. An alternative intermediary plugin framework, iPlug2[29], adds Web Audio as a target via the Web Audio Modules[4] standard but is also implemented as a C++ API. Although both JUCE and iPlug2 provide the ability to address multiple targets from a single codebase, they still require separate compilation and binary distribution

for each platform.

Another approach can be observed in the use of domain-specific languages (DSLs) for audio programming. Textual examples include CSound[31], FAUST[37], and CMajor[19]; visual examples include Max[9], Gen˜[51], and PureData[40]. The primary advantages of these languages are that they enable the developer to focus strictly on audio processing logic without handling memory management and other low-level programming tasks, and that they often target multiple deployments by exporting generated code. Some downsides are that they require programmers to learn a new language, that they may constrain the developer to a particular paradigm of signal processing, and that some are not free or open-source.

The ability of DSLs and intermediary plugin frameworks to target multiple deployment environments can be visualized as seen in Figure 2.1. They are single-input, multiple output systems.

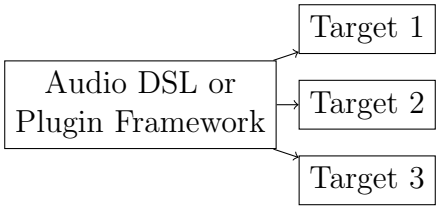

Figure 2.1: *Schematic representation of an audio DSL or plugin framework targeting multiple deployment environments.*

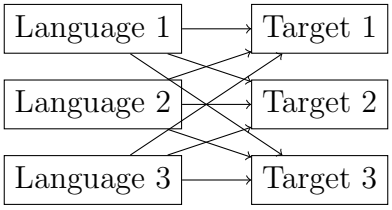

Figure 2.2: *Schematic representation of per-language, per-target toolchains.*

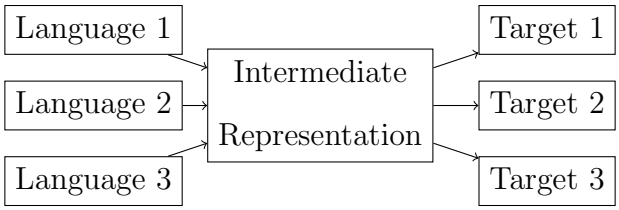

Figure 2.3: *Schematic representation of multiple languages targeting an intermediate representation that supports multiple deployments.*

A more flexible approach is a solution that can receive input in the form of source code from myriad programming languages, and also target multiple deployment environments. Traditionally, this may look like Figure 2.2, which depicts how each programming language must have its own toolchain to target each deployment environment.

Using an intermediate representation, the complexity of a multi-input, multi-output system may be reduced. A runtime for the intermediate format in any deployment environment enables all programming languages that target the format to immediately target that deployment. In turn, any programming language that can compile to the representation is able to target any environment that supports a suitable runtime. This is visualized in Figure 2.3.

2.2 WebAssembly in IoT Development

The rapid growth of the Internet of Things (IoT) has intensified the need for development models that decouple application logic from hardware-specific implementation details[38]. Like the world of embedded audio, IoT deployments are characterized by extreme heterogeneity: devices vary widely in processor architecture, memory constraints, operating systems, and peripheral configurations. Traditionally, this diversity has required application developers to maintain separate codebases or heavily conditional builds for each target, increasing complexity and reducing portability.

To enable easier development and maintain of IoT software, streamlined multi-target

development paradigms have emerged. These approaches enable software components written in different programming languages or otherwise originating from different toolchains (multi-source) to be deployed across diverse hardware platforms (multi-target) with minimal modification. Central to this model is the use of intermediate representation (IR), virtual execution layers, or other means that abstract away underlying architectural details. By targeting an IR, developers can focus on application logic and offload platform-specific implementation details to runtimes or translation layers designed to deterministically handle final deployment from a portable distributable.

WebAssembly (WASM) has gained attention in IoT research as such a portable, flexible IR[27][50][44]. Its well-defined execution semantics and compact binary format make it suitable for constrained devices, and toolchains exist to produce WASM from multiple source languages, including C/C++, Go, and Rust, enabling language-agnostic development. On the target side, WASM modules can be interpreted, just-in-time (JIT) compiled, or ahead-of-time (AOT) translated to native code, offering flexibility in runtime architecture.

This multi-source, multi-target model leveraging WASM reduces duplication of effort, improves maintainability, and enables modular software distribution across heterogeneous embedded systems. Its application to real-time audio DSP extends these principles to a domain with strict latency and determinism requirements, where portability has historically been limited by tight coupling between algorithms and hardware-specific toolchains.

2.3 Existing Approaches to Third-Party Plugins in Embedded Audio

Third-party plugins for embedded audio systems have been explored in industry, hobbyist and research contexts, but mass adoption is yet to occur.

One approach is use of the LV2 audio plugin standard, a format specific to Linux.

A major advantage of using LV2 is that it is available as a JUCE target, so existing JUCE plugins can be ported to new Linux platforms with a conditional build for the host architecture. Some downsides of this approach are that an operating system is required, and that most existing JUCE plugins do not have graphical user interfaces (GUIs) tailored to embedded systems. Examples that use the LV2 approach such as the MOD Audio ecosystem[35] and Darkglass Anagram[10] encourage plugin developers to make custom GUIs for each platform.

Figure 2.4: *The Darkglass Anagram, a commercial product that supports LV2 plugins*

An issue facing systems that depend on LV2 is that users are beholden to developers of their favorite plugins to produce and publish binaries targeting their particular hardware, and most vendors of popular audio plugins do not publish LV2 binaries. A unique approach for porting commercial plugins to Linux systems that bypasses this problem can be studied in [20]. Using `yabridge`[18], a compatibility layer for running Windows audio plugin binaries on Linux, this hobbyist demonstrated how popular VST plugins can be used on Linux systems. The major disadvantage is that performance degradation was shown to be 50%. They did not explicitly demonstrate Windows VSTs running on *embedded* Linux, but the primary problem of adapting user interface by connecting to a remote GUI or physical controls could be resolved in the implementation of the plugin host.

Another approach is one presented by researchers in [46], who implemented a novel framework for developing embedded audio applications that expect the GUI and audio processing to be separated over network. This approach relies on a custom operating

system they maintain, called EarOS.

While not explicitly related to plugins, researchers in [28] developed an open-source framework for deploying the same audio processing to heterogeneous embedded systems. Their support for third-party audio processing defined in C or MATLAB/Simulink provides an example of coexistence between embedded audio modules authored in different programming languages.

Chapter 3

Performance Benchmarking

To investigate the feasibility of WebAssembly (WASM) as a universal distributable for embedded audio applications, I have compiled four representative audio processing routines to WASM with a consistent binary interface using Emscripten[8] and measured execution time of each across three WASM runtimes on a resource contained target. For a baseline reference, I compiled and benchmarked each of the four routines as native ARM code as well via `gcc-arm-none-eabi` invoked with an optimization level of `-Ofast`.

3.1 Algorithms

In order to contextualize this research relative to real-world applications, I sourced a set of DSP algorithms that span major paradigms in both contemporary audio effects design and development workflows.

3.1.1 FAUST Reverb

The FAUST domain-specific programming language (DSL) is popular among audio developers for its capability to generate code for many different targets to be compiled separately, as well as its high-level semantics for describing audio DSP. The maintainers

of FAUST supply many example programs, including a reverb algorithm I have used here. The algorithm uses physical modeling techniques to emulate a spring reverb effect.

3.1.2 Gen˜ Phaser

Gen˜ is another popular domain-specific programming language (DSL) for audio. A major difference from FAUST is that Gen˜ is a *visual* programming language, providing a user interface that is often less challenging for beginners. Like many other audio DSLs, Gen˜ provides the ability to export code to target different platforms. For the purposes of benchmarking, I used Gen˜ to construct an elementary phaser effect with four cascaded state variable filters (from the GO library[52]) and a low-frequency oscillator.

3.1.3 Klon Centaur

The Klon Centaur is a highly acclaimed guitar overdrive pedal designed in the 1990s, whose circuitry has been copied in analog and digital forms many times over[7]. Prominent audio DSP programmer and scholar Jatin Chowdhury maintains an open-source C++ implementation using wave digital filters, a popular technique in audio DSP for white-box modeling of analog circuits.

3.1.4 Neural Amp Modeler

Neural Amp Modeler (NAM)[2] is a free and open-source standard for black-box modeling of audio equipment using neural networks. It has become highly popular in both hobbyist communities and industry, but implementations vary widely across platforms. For this benchmark I have used a C++ implementation of a NAM inference engine authored by Keith Bloemer[3], written specifically for use on an embedded platform.

3.2 Runtimes

To compare deployment strategies for WASM-based audio DSP on an embedded target, three distinct execution pipelines were selected for evaluation. They represent three fundamentally different approaches to executing WASM modules: ahead-of-time (AOT) compilation paired with an embedded-focused runtime, static translation to C followed by conventional compilation, and pure interpretation.

3.2.1 Wasm Micro Runtime (WAMR)

The WebAssembly Micro Runtime (WAMR), maintained by the Bytecode Alliance[5], is a lightweight WASM runtime explicitly designed for embedded systems. It supports multiple execution modes, including interpretation and AOT translation. For this study, WAMR was used in AOT mode, in which WASM bytecode is compiled offline into target-specific native machine code. The resulting artifact is managed by the WAMR runtime, which provides memory management, stack handling, and host function bindings.

3.2.2 wasm2c

`wasm2c` is a tool provided in the WebAssembly Binary Toolkit (WABT)[49] that translates WASM modules into C source code. Unlike WAMR, `wasm2c` is not a "runtime" in the traditional sense; it is a static translation step. I have leveraged a thin "runtime" implemented by wasm3labs[53] to manage exported modules.

3.2.3 wasmi

`wasmi`[14] is a pure WebAssembly interpreter written in Rust. It executes WASM bytecode directly using a stack-based virtual machine, without just-in-time (JIT) or AOT compilation. Wasmi was selected to represent a baseline interpreter-only deployment strategy, providing contrast to AOT-based approaches by illustrating the performance

cost of direct bytecode interpretation.

3.3 Embedded Target

I have chosen the Electrosmith Daisy Seed as a representative platform for benchmarking embedded processing. Daisy is a widely popular embedded platform, similar to the Arduino[1] ecosystem but with a focus on audio. The STM32 processor at its core is widely used across research and industry for embedded audio systems[12][39][26].

3.4 Benchmarking Code

To benchmark execution time, the STM32H7's TIM2 hardware counter is leveraged to measure elapsed time in both raw tick counts and microseconds. Timer `start()` and `stop()` calls bookend calls to the WASM runtimes to process 128-sample blocks of audio data. For each algorithm and runtime combination, the elapsed time is averaged over 100 iterations. A 400MHz clock rate is used. The source code for benchmarks on WAMR, `wasm2c`, and `wasmi` are available at [23], [24], and [25], respectively, with dedicated Git branches for benchmarking each algorithm. Native benchmarking is included in `wasm2c -demo`. Quantitative results are presented in Table 3.1, Table 3.2, and Figure 3.1.

Table 3.1: *Raw sample throughput (samples/sec) across runtimes.*
Cells shaded green represent algorithm/runtime combinations with throughput greater than 48kHz, indicating real-time performance

Algorithm	Native	WAMR AOT	Wasm2c	Wasmi
FAUST Reverb	76,141	67,288	65,222	6,259
Gen~ Phaser	345,862	189,599	215,242	974
Klon Centaur	142,487	32,880	38,030	–*
Neural Amp Modeler	76,759	5,565	22,237	–*

*Algorithm/runtime combination did not achieve stable performance

Table 3.2: *Relative performance comparison of different algorithms across runtimes.*

Algorithm	Native	WAMR AOT	Wasm2c	Wasmi
FAUST Reverb	1.0x	0.8837x	0.8566x	0.0822x
Gen˜ Phaser	1.0x	0.5482x	0.6223x	0.0028x
Klon Centaur	1.0x	0.2308x	0.2669x	–*
Neural Amp Modeler	1.0x	0.0725x	0.2897x	–*

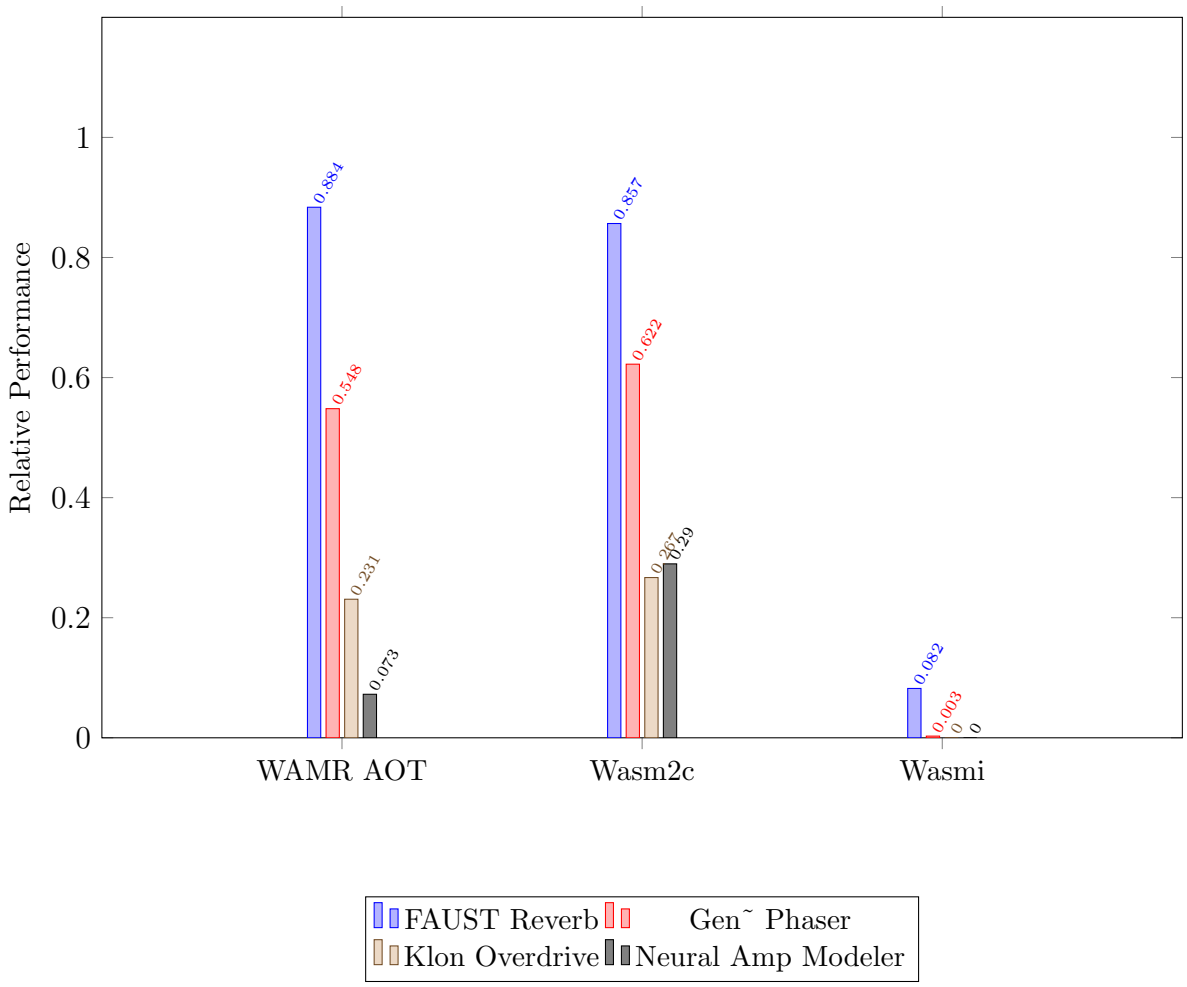

Figure 3.1: *Performance comparison relative to native (1.0x).*

Chapter 4

Developer Tooling

Providing great developer tooling can endow even the most anemic processor with outsized capabilities [47][13]. To promote the authoring and distribution of WASM-based audio DSP modules with a smooth developer experience (DX), I have created a proof-of-concept development environment that runs entirely client-side in the browser.

A contributor to the siloed nature of many embedded systems is their scattered development environments. To provide a glimpse into the status quo of the field circa 2005, I present a quote from [6]:

> When you're developing your embedded system, it is best to start with a development kit from the processor's manufacturer [...] it should [...] include a nice Integrated Development Environment (or IDE). [...]
>
> Some of these companies do a fantastic job, while others seem to employ stray chimpanzees as design engineers. [...]
>
> Almost without exception, the IDEs will run on only one platform and under only one operating system. No prizes for guessing which one. So, if your preferred environment is a Unix workstation, generally you're out of luck.

Fortunately for contemporary programmers, developer tooling for embedded systems

has become much more friendly. The advent of Arduino[1] demonstrated that the application of design principles from creative coding tools to embedded systems can fuel major innovation by enfranchising more developers.

Exemplary developer tooling geared towards enfranchisement of new programmers can be found in web-based IDEs, which have gained prominence in the past decade as highly accessible tools for people learning computer programming[48][11]. A major advantage of web-based IDEs is that they are consistent across operating systems, increasing ease of setup compared to local development tools. WASM-based audio DSP modules lend themselves well to a web-based IDE, as WASM produced by developers' code can be rapidly auditioned using the Web Audio API. Notable audio-focused web-based IDEs exist for the FAUST[43] and ChucK[36] audio DSLs, but not for C/C++, Rust, and other popular languages for audio.

In 2025 I began work on a web-based IDE for writing and auditioning C/C++ audio code in the browser, entitled Hear-C[22]. My initial architecture relied on a server for compiling C/C++ to WASM using Emscripten, but discovery of `wasm-clang`[45], a C/C++ to WASM compiler itself compiled to WASM, enabled me to remove dependency on a server.

Given the promising performance demonstrated by Web Assembly Micro Runtime (WAMR) in Chapter 3, I investigated the possibility of compiling its open-source `wamrc` compiler (which translates WASM to WAMR AOT modules) itself to WASM. This proved highly doable within a span of a few days leveraging agentic coding tools.

In a fork of Hear-C, I added the functionality to transpile WASM to WAMR AOT via the WASM-compiled `wamrc`, as well as tools for flashing and running the resulting modules on the Daisy Seed. Though I focused on the Daisy (WAMR AOT for ARM Cortex M7), the WASM-compiled `wamrc` I have published supports the full range of ARM Cortex M and A series processors that a standard build of `wamrc` does. I disabled other targets (`aarch64`, `x86_64`, `riscv32`, etc) via build system flags due to time constraints,

but expect that these would be relatively easy to add in the future. Source code for the web-based IDE can be found at [21].

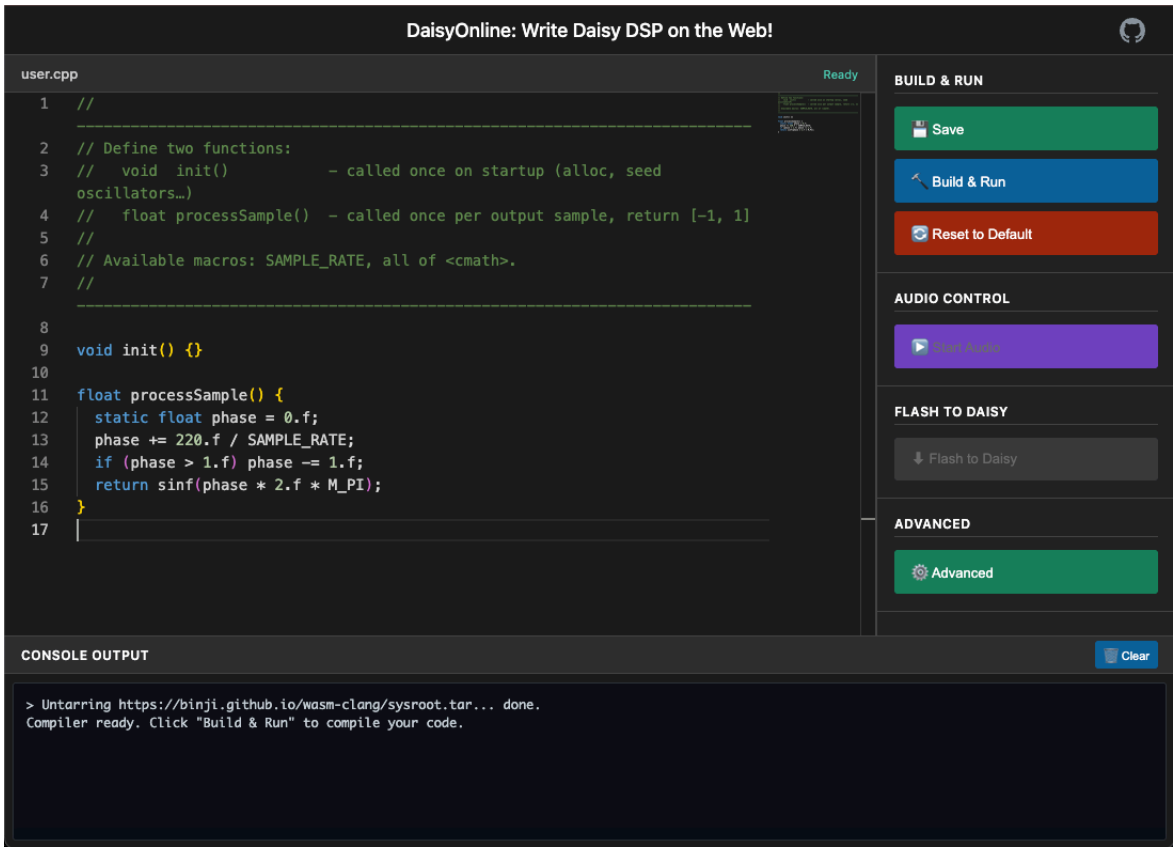

Figure 4.1: *DaisyOnline, an online IDE for developing and deploying WebAssembly on the Daisy Seed*

Chapter 5

Discussion

Performance benchmarking in Chapter 3 demonstrates that WebAssembly (WASM) can support real-time audio DSP on embedded hardware under certain conditions, but further research is needed to determine the bounds of its performance.

Across the four tested algorithms, interpreter-based execution (`wasmi`) did not achieve usable throughput and can be discarded as a runtime approach in future work. Ahead-of-time (AOT) compilation using WAMR and translation via `wasm2c` both yielded viable performance for algorithms whose memory footprint is known at compile time, but demonstrated highly variable success with algorithms that do not.

Two patterns are evident. First, strategies that produce native machine code at flash time are markedly more suitable for real-time audio than runtime interpretation of bytecode. Second, runtime overhead is highly workload-dependent. Algorithms that rely heavily on floating-point arithmetic but maintain predictable control flow and limited dynamic allocation are more amenable to WASM deployment.

It is important to note that these measurements were obtained without aggressive runtime customization. Further opportunities for performance gains exist, including better leveraging of host-imported math intrinsics, improved compiler pipelines, and tighter ABI and runtime specialization. The results presented in this work therefore

represent a lower bound on achievable performance.

A reasonable question remains: if real-time performance of WASM-based DSP modules relies on AOT translation, can they truly be considered a universal distributable? The author argues yes. I encourage conception of the AOT compiler as part of the runtime or flash tool for each target, a responsibility laid on hardware manufacturers looking to increase adoption of their product rather than a burden on audio plugin developers. In some cases, AOT compilation for embedded systems can even occur on the target device, as seen in [32].

Chapter 6

Conclusion

This project documents investigation of WebAssembly (WASM) as a hardware-agnostic intermediate representation for distributing audio DSP modules to embedded targets. By benchmarking three execution strategies on a representative embedded platform and creating proof-of-concept developer tooling, I have evaluated the practicality of WASM-based processing in resource-constrained audio systems and presented an example workflow for development and distribution.

Performance measurements indicate that interpreter-based execution is not suitable for real-time embedded DSP. However, approaches leveraging ahead-of-time (AOT) translation mechanisms demonstrate acceptable performance for a meaningful subset of audio algorithms.

These findings support the broader claim that a carefully specified WASM ABI for audio plugins could form the basis of a universal DSP distribution format, similar to that proposed in [15]. Such a format would allow algorithms written in multiple programming languages to compile to a single intermediate representation and be uniformly deployed across embedded devices, desktop applications, and web platforms using appropriate runtimes.

The primary contributions of this work are: (1) the exploration of multiple execution

strategies for WASM-based audio DSP modules, (2) empirical benchmarking of their relative performance on representative embedded hardware, and (3) creation of web-based developer tooling to facilitate their authoring and deployment. Future work will include refinement of a WASM ABI for audio plugins, further exploration of compiler and runtime optimizations, and evaluation on additional deployment targets.

Bibliography

[1] Arduino. *Arduino Official Website*. www.arduino.cc. Accessed: March 07 2026.

[2] Steven Atkinson. *Neural Amp Modeler*. https://www.neuralampmodeler.com/. 2024.

[3] Keith Bloemer. *Mercury: Neural Amp Modeler Based Amp Emulator*. https://github.com/GuitarML/Mercury. GitHub repository. Accessed: Mar. 7, 2026. 2024.

[4] Michel Buffa et al. "Web Audio Modules 2.0: An Open Web Audio Plugin Standard". In: *Companion Proceedings of the Web Conference 2022*. WWW '22. Virtual Event, Lyon, France: Association for Computing Machinery, 2022, 364–369. ISBN: 9781450391306. DOI: 10.1145/3487553.3524225. URL: https://doi.org/10.1145/3487553.3524225.

[5] Bytecode Alliance. *Bytecode Alliance*. https://bytecodealliance.org/. Accessed: Mar. 10, 2026. 2026.

[6] John Catsoulis. *Designing Embedded Hardware: Create New Computers and Devices*. O'Reilly Media, Inc., 2005.

[7] Jatin Chowdhury. "A comparison of virtual analog modelling techniques for desktop and embedded implementations". In: *arXiv preprint arXiv:2009.02833* (2020).

[8] Emscripten Contributors. *Emscripten Documentation*. https://emscripten.org/docs/index.html. 2015. (Visited on 03/05/2026).

[9] Cycling '74. *Max/MSP/Jitter*. Version 8.6. Cycling '74. 2024. URL: https://cycling74.com.

[10] Darkglass Electronics. *Plugin-Dev-Setup*. https://github.com/Darkglass-Electronics/Plugin-Dev-Setup. GitHub repository. Accessed: Mar. 7, 2026. 2026.

[11] Joshua Delgadillo and Chris Piech. *A Fully Client-Side C++ IDE for CS1 Using WebAssembly*. https://cs191.stanford.edu/projects/Delgadillo2025.pdf. Accessed: 2026-03-09. 2025.

[12] emilyharpist2. *What pedal makers don't want you to see*. https://www.youtube.com/watch?v=WRcm2b877ek. YouTube video, accessed 5 Mar 2026. 2026.

[13] Ron Evans. *The Smallest Thing That Could Possibly Work: WASM on Microcontrollers With TinyGo*. https://www.youtube.com/watch?v=24NDl27mZu4. YouTube video. Accessed: Mar. 8, 2026. 2024.

[14] Robin Freyler. *Wasmi - WebAssembly (Wasm) Interpreter*. https://github.com/wasmi-labs/wasmi. GitHub repository. Accessed: March 5, 2026. 2018.

[15] Benedict Gaster and Max Cole. "Audio Anywhere with Faust". In: *Proceedings of the 2nd international faust conference*. 2020.

[16] Andreas Haas et al. "Bringing the web up to speed with WebAssembly". In: *SIGPLAN Not.* 52.6 (June 2017), 185–200. ISSN: 0362-1340. DOI: 10.1145/3140587.3062363. URL: https://doi.org/10.1145/3140587.3062363.

[17] Mislav Has et al. "WebAssembly on Resource-Constrained IoT Devices: Performance, Efficiency, and Portability". In: *IoT Workshop Proceedings* 1 (1 Nov. 2025), pp. 29–35. ISSN: 3061-1008. DOI: 10.34749/3061-1008.2025.5. URL: https://iotw-proceedings.dsg.tuwien.ac.at/article/id/753/.

[18] Robbert van der Helm. *yabridge: A Modern Way to Use Windows VST2, VST3 and CLAP Plugins on Linux*. https://github.com/robbert-vdh/yabridge. GitHub repository. Accessed: Mar. 7, 2026. 2026.

[19] Stephanie Huang et al. "C-Major Language Reference Manual". In: ().

[20] Interfacing Linux. *Windows VSTs on Linux with yabridge*. https://www.youtube.com/watch?v=u6J6nQyA3ug. YouTube video. Accessed: Mar. 7, 2026. 2024.

[21] Joel A. Jaffe. *DaisyOnline*. https://github.com/jaffco/DaisyOnline. GitHub repository. Accessed: 5 Mar 2026. 2026.

[22] Joel A. Jaffe. *Hear-C*. https://github.com/joeljaffesd/Hear-C. GitHub repository. Accessed: 5 Mar 2026. 2026.

[23] Joel A. Jaffe. *wamr-demo*. https://github.com/jaffco/wamr-demo. GitHub repository. Accessed: 5 Mar 2026. 2026.

[24] Joel A. Jaffe. *wasm2c-demo*. https://github.com/jaffco/wasm2c-demo. GitHub repository. Accessed: 5 Mar 2026. 2026.

[25] Joel A. Jaffe. *wasmi-demo*. https://github.com/jaffco/wasmi-demo. GitHub repository. Accessed: 5 Mar 2026. 2026.

[26] Thitaphan Jongsataporn and Sorawat Chivapreecha. "Embedded hardware implementation of noise gate on STM32 processor using MATLAB/Simulink". In: *2024 International Conference on Photonics Solutions (ICPS2024)*. Vol. 13518. SPIE. 2025, pp. 103–110.

[27] István Koren. "A Standalone WebAssembly Development Environment for the Internet of Things". In: *Web Engineering*. Ed. by Marco Brambilla et al. Cham: Springer International Publishing, 2021, pp. 353–360. ISBN: 978-3-030-74296-6.

[28] Patrick Lai. "AudioReach: An End-to-End Solution". In: *Proceedings of the 19th Linux Audio Conference*. 2025.

[29] Oliver Larkin, Alex Harker, and Jari Kleimola. "iPlug 2: Desktop Plug-in Framework Meets Web Audio Modules". In: *Proceedings of the International Web Audio Conference*. Ed. by Jan Monschke et al. WAC '18. Berlin, Germany: TU Berlin, 2018.

[30] C. Lattner and V. Adve. "LLVM: a compilation framework for lifelong program analysis & transformation". In: *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 2004, pp. 75–86. DOI: 10.1109/CGO.2004.1281665.

[31] Victor Lazzarini et al. *Csound: a sound and music computing system*. Springer, 2016.

[32] Borui Li et al. "WaWoT: Towards Flexible and Efficient Web of Things Services via WebAssembly on Resource-Constrained IoT Devices". In: *IEEE Transactions on Computers* 74.3 (2025), pp. 1094–1108. DOI: 10.1109/TC.2024.3500385.

[33] Clifford Liem et al. "Am embedded system case study: the firm ware development environment for a multimedia audio processor". In: *Proceedings of the 34th Annual Design Automation Conference*. DAC '97. Anaheim, California, USA: Association for Computing Machinery, 1997, 780–785. ISBN: 0897919203. DOI: 10.1145/266021.266373. URL: https://doi.org/10.1145/266021.266373.

[34] Tim Lindholm et al. *The Java virtual machine specification*. Addison-wesley, 2013.

[35] MOD Audio. *The MOD Platform*. https://mod.audio/platform/. Accessed: 2026-03-07. 2026.

[36] Michael Mulshine et al. "WebChucK: Computer Music Programming on the Web." In: *NIME*. 2023.

[37] Yann Orlarey, Dominique Fober, and Stéphane Letz. "FAUST : an Efficient Functional Approach to DSP Programming". In: *NEW COMPUTATIONAL PARADIGMS*

FOR COMPUTER MUSIC. Ed. by Editions DELATOUR FRANCE. 2009, pp. 65–96. URL: https://hal.science/hal-02159014.

[38] Pankesh Patel and Damien Cassou. "Enabling high-level application development for the Internet of Things". In: *Journal of Systems and Software* 103 (2015), pp. 62–84. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2015.01.027. URL: https://www.sciencedirect.com/science/article/pii/S0164121215000187.

[39] Diamond Pedals. *Memory Lane Product Page*. https://www.diamondpedals.com/products/memory-lane. 2025. (Visited on 03/05/2026).

[40] Miller S. Puckette. "Pure Data". In: *Proceedings of the International Computer Music Conference (ICMC '97)*. International Computer Music Association. Thessaloniki, Greece, 1997, pp. 224–227.

[41] Raw Material Software Ltd. *JUCE Framework*. Accessed: 2026-02-23. 2026. URL: https://juce.com/.

[42] Partha Pratim Ray. "An Overview of WebAssembly for IoT: Background, Tools, State-of-the-Art, Challenges, and Future Directions". In: *Future Internet* 15.8 (2023). ISSN: 1999-5903. DOI: 10.3390/fi15080275. URL: https://www.mdpi.com/1999-5903/15/8/275.

[43] Shihong Ren et al. "FAUST online IDE: dynamically compile and publish FAUST code as WebAudio Plugins". In: *WAC 2019-5th Web Audio Conference*. 2019.

[44] Guru Pramod Rusum. "WebAssembly across Platforms: Running Native Apps in the Browser, Cloud, and Edge". In: *International Journal of Emerging Trends in Computer Science and Information Technology* 3.1 (2022), 107–115. DOI: 10.63282/3050-9246.IJETCSIT-V3I1P112. URL: https://ijetcsit.org/index.php/ijetcsit/article/view/341.

[45] Ben Smith. *wasm-clang*. https://github.com/binji/wasm-clang. GitHub repository. Accessed: Mar. 8, 2026. 2019.

[46] Gary Spittle, Michael Lee, and Weiming Li. "Developing plugins for your ears". In: *Audio Engineering Society Convention 151*. Audio Engineering Society. 2021.

[47] The New York Times. *The Monster-Slaying Game You Can Play Almost Anywhere*. https://www.nytimes.com/2025/05/21/arts/play-doom-ports.html. Accessed: 2026-03-06. 2025.

[48] Hai T. Tran et al. "An interactive Web-based IDE towards teaching and learning in programming courses". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. 2013, pp. 439–444. DOI: 10.1109/TALE.2013.6654478.

[49] WABT Contributors. *WABT: The WebAssembly Binary Toolkit*. https://github.com/WebAssembly/wabt. GitHub repository. Accessed: 5 Mar 2026.

[50] Linus Wagner et al. "On the Energy Consumption and Performance of WebAssembly Binaries across Programming Languages and Runtimes in IoT". In: *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. EASE '23. Oulu, Finland: Association for Computing Machinery, 2023, 72–82. ISBN: 9798400700446. DOI: 10.1145/3593434.3593454. URL: https://doi.org/10.1145/3593434.3593454.

[51] Graham Wakefield. *Real-time meta-programming for interactive computational arts*. University of California at Santa Barbara, 2012.

[52] Graham Wakefield and Gregory Taylor. *Generating sound & organizing time : thinking with gen book 1*. [Cycling '74], 2022. URL: https://ci.nii.ac.jp/ncid/BD0831917X.

[53] Wasm3 Labs. *embedded-wasm-apps*. https://github.com/wasm3/embedded-wasm-apps. GitHub repository. Accessed: 5 Mar 2026. 2021.